Enterprise AI Reference Architecture

A Companion to PAHA and Modus Primus

Structural decomposition for enterprise and systems architects

Version 1.1

James (JD) Longmire
Northrop Grumman Fellow (unaffiliated research)
Chief Architect, Digital Ecosystems
ORCID: 0009-0009-1383-7698
Correspondence: jdlongmire@outlook.com

February 2026

Related Architectural Framework
PAHA Rev 2.2: doi:10.5281/zenodo.20112631
Modus Primus v1.1: doi:10.5281/zenodo.20113785

1. Introduction

1.1 Purpose and Audience

This reference architecture provides a structural decomposition that enterprise and systems architects can use to design, evaluate, and instantiate AI-enabled enterprise capabilities. It identifies the components, their responsibilities, their relationships, and the principles that govern their composition. It is intentionally practical: the architect should be able to use this document to draw their organization's specific architecture, identify component owners, and plan adoption sequencing.

The primary audience is enterprise architects and systems architects working in regulated industries, defense IT, and sovereignty-constrained enterprises where vendor-bundled AI orchestration cannot meet the federation, governance, and assurance requirements that the operational context imposes. The secondary audience is technology leadership, governance councils, and program managers responsible for AI capability delivery.

This document is not a vendor implementation guide. It does not endorse specific cloud providers, foundation models, orchestration platforms, or governance toolchains. Implementation choices are properly enterprise-specific and are captured in per-enclave technical baselines, not in the canonical reference architecture.

1.2 Relationship to PAHA and Modus Primus

This reference architecture is the third document in a complementary family.

PAHA (Portable Agent Harness Architecture) specifies the architectural pattern. It states what an architecture for governed, sovereignty-bounded AI must achieve: a durable meta-harness providing centralized governance, bounded execution, and substrate arbitration over fit-for-purpose consoles and composable agents. PAHA operates at the level of architectural commitments.

Modus Primus specifies the engineering practice. It states how the enterprise's engineering function produces and maintains a PAHA-conformant architecture: the Work Breakdown Structure, the five-tier federation model, the V&V instruments, the means and mechanisms selection relation, the trust escalation model. Modus Primus operates at the level of engineering specifications.

This document specifies the structural decomposition. It states what the components actually are, what each does, how they relate, and what principles govern their composition. The reference architecture operates at the level of instantiation: an architect should be able to use this document to draw the boxes and lines for a specific enterprise deployment.

Each document operates at a different level of abstraction. Together they form a complete framework. The relationships are:

PAHA primitive	This reference architecture	Modus Primus
Meta-harness (durable architectural layer)	Layer 1 (Enterprise AI Orchestrator) and parts of Layer 2A (Shared Services)	WBS Section 3 (Technical Baseline)
Three-plane decomposition (governance, cognitive, operational)	Cross-cutting Governance, Layer 2A Shared Services, Layer 3 Runtime	WBS layer architecture (Appendix B)
Seven minimum viable harness services	Layer 2A core services	Mechanism Layer (B.7)
Enclave constraint and federation pattern	Section 10 (Federation Across Enclaves)	Five Modus tiers (Section 4)
Capability registry (purposive election)	Layer 2A Capability Registry, Section 9.5 (Means Election pattern)	Means and mechanisms selection (Section 2, Appendix C)
Trust escalation as governance primitive	Cross-cutting Governance, Section 9.3 (Human Approval Cycle)	Trust escalation model (Sections 2.5, 3.7.1)

Readers who want the architectural pattern in full should consult PAHA. Readers who want the engineering practice for producing it should consult Modus Primus. This document is the bridge: structural enough to design against, principled enough to remain valid as implementations evolve.

1.3 How to Use This Document

The architecture is presented as five sections plus three diagrams. The recommended reading sequence depends on the architect's purpose.

For initial framework comprehension, read Sections 2 through 8 in order. The foundational principles in Section 2 prepare the reader for the layer-by-layer treatment that follows. The cross-cutting governance discussion in Section 8 closes the structural decomposition.

For design work, supplement the layer treatments with the interaction patterns in Section 9. The patterns show how the components actually work together for specific scenarios; the layer descriptions show what the components are. Design work needs both.

For sovereignty-constrained enterprises, Section 10 (Federation Across Enclaves) is foundational. The single-enclave description in earlier sections is incomplete without it.

For implementation planning, Section 11 (Adoption Sequencing) identifies which components an enterprise builds first, what advances to subsequent phases, and what triggers progression. An architecture without adoption guidance is paperware.

---

2. Foundational Principles

The reference architecture is governed by seven principles. These principles operate as decision filters: when an architect encounters a design choice not anticipated by the architecture, the principles should guide resolution.

1. The Orchestrator coordinates; it does not execute. Layer 1 (Enterprise AI Orchestrator) routes requests, enforces policy, and coordinates workflows. It does not become a mega-agent that handles every interaction itself. Coordination is structurally separated from execution.

2. Every domain consumes shared services. Layer 2A (Shared Enterprise AI Services) is the single source of identity, memory, retrieval, model gateway, evaluation, and observability. Domains do not instantiate their own identity infrastructure, their own retrieval pipelines, or their own evaluation harnesses. Duplication is a smell, not an optimization.

3. Every action passes through the Runtime. Layer 3 (AI Runtime and Execution) is the single point of policy enforcement before real-world consequence. Pre-action validation, sandbox enforcement, resource limits, and audit emission are properties of the Runtime, not optional behaviors of individual agents. Bypassing the Runtime is bypassing the architecture.

4. Every tool is a mechanism until purposively elected as a means. A tool added to the Mechanism Layer is operationally available but not part of the system's disposition. Election to means status is a governance act, not a procurement event. This prevents silent capability creep across the architecture.

5. Every action produces audit evidence. Audit traceability is an architectural commitment, not an operational policy. The Runtime emits structured evidence at every step; the Observability service captures it; the Governance layer queries it. The audit chain is queryable end-to-end.

6. Every enclave hosts its own instance. Federation between security enclaves occurs through schema and signal aggregation, not through cross-enclave invocation. Each enclave is a self-contained reference architecture deployment. This is the only pattern compatible with sovereignty constraints.

7. The Declare-Enforce-Evidence Doctrine. Governance declares; Runtime enforces; Observability evidences. Policy authority lives in the Governance layer. Policy enforcement happens in the Runtime. Evidence of enforcement is captured by the Observability service. Each layer has a distinct role in the audit lifecycle; conflating them produces architectures that look governed but are not. This doctrine is named

because the rest of this document refers back to it: when an architect faces a design choice involving governance, enforcement, or audit evidence, the doctrine specifies which layer owns the concern.

These principles are not derived from this architecture; they encode the PAHA architectural commitments and the Modus Primus engineering discipline in operational form. An architecture that violates any of these principles is not PAHA-conformant, regardless of what its component diagrams look like.

---

3. Architecture at a Glance

The reference architecture comprises five sections organized in three dependency layers plus a cross-cutting concern.

Layer 1 – Coordination. The Enterprise AI Orchestrator. The single coordination plane for the architecture.

Layer 2 – Capability. Two sections: Shared Enterprise AI Services (the common services every domain consumes) and Domain AI Ecosystems (the bounded scopes where domain-specific agents and tools operate). The two sections of Layer 2 have a peer relationship: domains consume shared services; shared services are accessed by domains.

Layer 3 – Foundation. The AI Runtime and Execution Layer. The foundation that every action passes through. Every layer above depends on Layer 3 for execution.

Cross-cutting – Governance, Risk, and Assurance. Controls that apply to every layer. Governance is not a layer in the dependency stack; it is a cross-cutting concern that declares policy enforced by the Runtime, evidenced by the Observability service, and inherited by every action.

The dependency rule is straightforward: Coordination depends on Capability, Capability depends on Foundation. An architect designing a domain ecosystem assumes the Shared Services are in place; an architect designing a Shared Service assumes the Runtime is in place. Governance applies to all of them.

Figure 1 shows the component view.

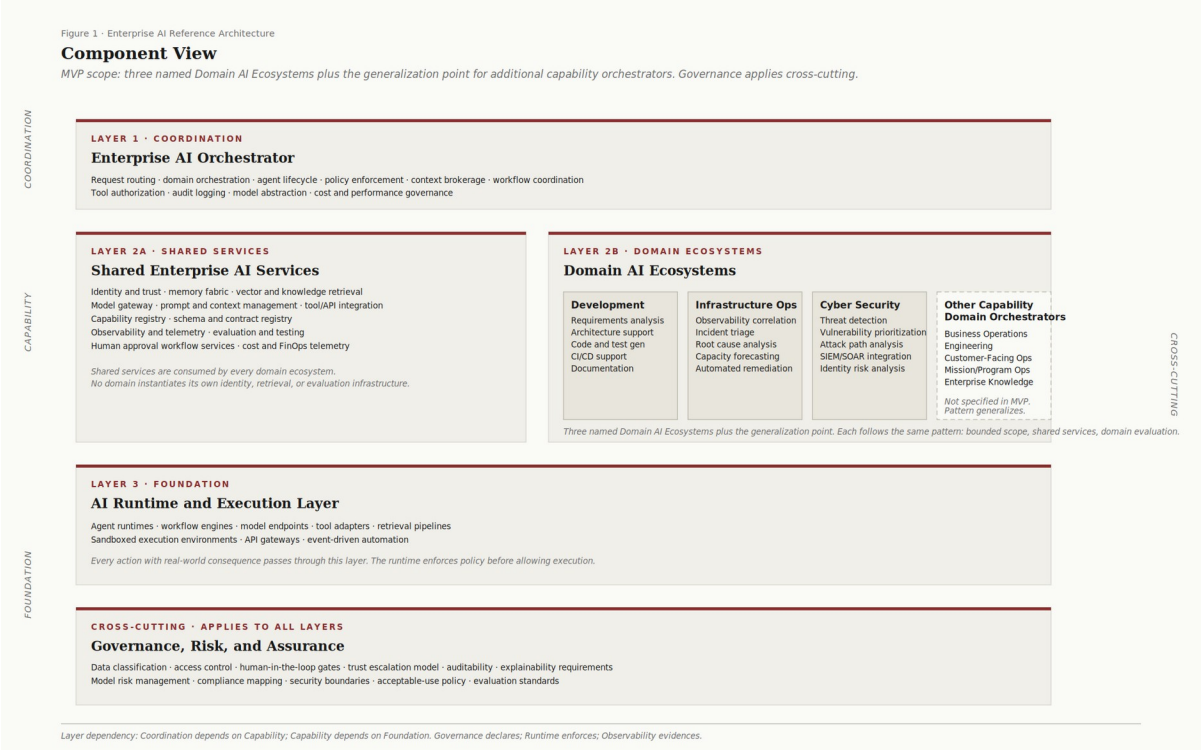

Figure 1. Enterprise AI Reference Architecture, Component View (MVP). Three named Domain AI Ecosystems plus the generalization point for additional capability orchestrators. Governance applies cross-cutting.

---

4. Layer 1 – Enterprise AI Orchestrator

The Enterprise AI Orchestrator is the coordination plane of the architecture. It is the single component responsible for routing requests, coordinating workflows, enforcing top-level policy, and managing agent lifecycles. Every coordinated enterprise AI workflow is mediated by the Orchestrator. The Orchestrator is the coordination plane; low-level runtime operations enforce policy directly through Layer 3 without re-traversing Layer 1. This distinction matters because the Orchestrator is not a chokepoint on every operation; it is the authority for workflow-level coordination, while the Runtime is the authority for action-level enforcement.

4.1 Responsibilities

The Orchestrator's primary responsibilities are:

- **Request routing.** Requests from fit-for-purpose consoles enter the Orchestrator, which determines which domain ecosystem should handle the request based on its scope, context, and authorization.

- **Domain orchestration.** Cross-domain workflows are coordinated by the Orchestrator. When a request requires action across multiple domains, the Orchestrator coordinates the sequence and handles inter-domain context passing.
- **Agent lifecycle management.** Persistent specialist agents (within domain ecosystems) and ephemeral agents (invoked for bounded tasks) are managed by the Orchestrator. Lifecycle includes instantiation, role binding, context scoping, and termination.
- **Policy enforcement at the coordination layer.** Policy decisions that apply across domains live with the Orchestrator. Domain-specific policy decisions remain within domain ecosystems. The Orchestrator does not duplicate domain policy; it composes cross-domain policy.
- **Context brokerage.** Context relevant to a request (user identity, session state, prior interactions, classification level) is brokered by the Orchestrator. The Orchestrator is the trusted authority for context that crosses component boundaries.
- **Workflow coordination.** Multi-step workflows that span multiple agents or domains are coordinated by the Orchestrator. The Orchestrator is the workflow engine for cross-domain processes.
- **Tool authorization.** The Orchestrator validates that the requesting agent is authorized to invoke the requested tool before forwarding the request to the Runtime. Tool authorization decisions reference the Capability Registry.
- **Audit logging.** The Orchestrator emits audit evidence at the coordination level: request received, routing decision, policy check, completion, outcome. This complements the lower-level audit evidence emitted by the Runtime.
- **Model abstraction.** The Orchestrator provides a stable interface to underlying foundation models, isolating consumers from substrate volatility. Model substitution is an architectural event that the Orchestrator absorbs, not an outage that propagates to consumers.
- **Cost and performance governance.** The Orchestrator enforces cost and performance budgets at the coordination level: per-domain budgets, per-user budgets, per-workflow budgets. The Shared Services layer provides cost telemetry; the Orchestrator applies governance against the telemetry.

4.2 What the Orchestrator Is Not

The Orchestrator coordinates; it does not execute. Specifically:

- The Orchestrator is not a mega-agent that handles every interaction itself. Domain-specific work happens in domain agents within their bounded ecosystems. The Orchestrator routes; it does not perform domain work.
- The Orchestrator is not a tool runtime. Tool execution happens in Layer 3 (the Runtime), not in the Orchestrator. The Orchestrator authorizes tool invocation; the Runtime executes it.

- The Orchestrator is not a model serving layer. Model inference happens in Layer 3 through the Model Gateway shared service. The Orchestrator brokers model selection; it does not perform inference.
- The Orchestrator is not a memory store. Memory persistence and retrieval happen in the Memory Fabric shared service. The Orchestrator brokers context; it does not store it.

The discipline matters because Orchestrators that drift into execution become bottlenecks, single points of failure, and accidental mega-agents that violate the bounded execution principle. The Orchestrator's value is in coordination; its failure mode is in execution.

---

5. Layer 2A – Shared Enterprise AI Services

Shared Enterprise AI Services are the common infrastructure that every domain consumes. The discipline is consistent: no domain instantiates its own identity service, its own memory store, its own evaluation harness. Duplication is prohibited.

5.1 Core Services

The minimum viable set of shared services:

- **Identity and trust.** Authentication and authorization for users, agents, and inter-component requests. Integrated with the enterprise IAM infrastructure. Provides the identity context that every audit record references.
- **Memory fabric.** Context persistence and retrieval across sessions, agents, and domains. Includes short-term conversational memory, long-term knowledge memory, and audit-history memory. Retention policies are governance-declared and enforced by the fabric.
- **Vector and knowledge retrieval.** Semantic search over enterprise content (documents, knowledge bases, runbooks, policy artifacts). Provides the grounding substrate for retrieval-augmented agents across domains.
- **Model gateway.** Unified access to foundation models. Abstracts model-specific APIs into a stable interface. Enforces model selection policy (which models are authorized for which classifications, which domains, which trust levels). Captures inference telemetry for cost and performance governance.
- **Prompt and context management.** Centralized prompt template management, version control, and context assembly. Prompts are first-class artifacts subject to governance review, not embedded strings in agent code.
- **Tool and API integration.** Connectors for enterprise systems, third-party APIs, and external tools. Tools are inventoried here; they become means only through purposive election (see the Capability Registry).

- **Capability registry.** The catalog of all available tools, models, retrieval sources, and agents in the architecture. The registry distinguishes between operationally available mechanisms and purposively elected means. (See Section 5.2.)
- **Schema and contract registry.** Centralized authority for agent contracts, tool interface contracts, federation schemas, and event schemas. Schema versioning, compatibility rules, and migration paths are managed here.
- **Observability and telemetry.** Captures audit evidence, performance metrics, cost telemetry, runtime assurance signals, and operational events. Provides the query surface for governance review and operational support.
- **Evaluation and testing.** Quality criteria, test harnesses, and outcome scoring for agents and tools. Domain-specific evaluation criteria plug into this shared framework; domains do not build evaluation infrastructure from scratch.
- **Human approval workflow services.** The substrate for trust-escalation-level-2 and level-3 workflows where human approval is required. Provides the request, review, decision, and audit-emission machinery for human-in-the-loop interactions.
- **Cost and FinOps telemetry.** Per-request, per-agent, per-domain, per-user cost telemetry. Aggregated to budget controls in the Orchestrator and to governance reporting.

5.2 The Capability Registry

The Capability Registry deserves specific attention because it is the operational realization of the means and mechanisms selection relation from PAHA and Modus Primus.

The registry distinguishes between two states for every tool, model, agent, or retrieval source in the architecture:

- **Mechanism.** Operationally available capacity. Listed in the registry, accessible to the Runtime, but not yet authorized for use by any agent or workflow. A mechanism becomes part of the system's disposition only through election to means status.
- **Means.** A mechanism that has been purposively elected by the Governance layer as authorized for use within specific scopes (domains, agents, trust levels, classifications). Means are governance-declared; the registry tracks the declaration.

When a new tool is added to the Mechanism Layer (Layer 3), it appears in the registry as a mechanism. To become a means, it requires a means election review (see Section 9.5). The review evaluates whether the mechanism advances the mission, whether its election is consistent with morals (security and safety constraints), and whether the existing means inventory justifies the addition.

The registry is the architectural antidote to silent capability creep. Without it, every newly procured tool becomes part of the architecture by accretion. With it, every capability addition is a deliberate governance act.

6. Layer 2B – Domain AI Ecosystems

Domain AI Ecosystems are the bounded scopes within which domain-specific agents, tools, and workflows operate. Each domain ecosystem is a coherent capability area with its own scope, its own evaluation criteria, and its own operational concerns.

6.1 Common Pattern

Every domain ecosystem follows the same pattern:

- **Bounded scope.** The domain's responsibilities are defined and limited. The scope is what the domain does; what the domain does not do is equally well-defined.
- **Shared service consumption.** The domain consumes the Shared Enterprise AI Services. It does not duplicate them. Identity, memory, retrieval, evaluation, observability all come from the shared layer.
- **Domain-specific agents.** The domain hosts persistent specialist agents whose role bindings, capability authorizations, and audit obligations are scoped to the domain. Agent contracts (per Modus Primus Appendix E) specify the constraints.
- **Domain-specific tools.** Some tools are domain-specific (e.g., a code analysis tool for the Development domain, a SIEM connector for the Cyber Security domain). These tools are inventoried in the Capability Registry and elected to means status through the standard process.
- **Domain-specific evaluation criteria.** Quality criteria and outcome scoring are domain-specific. The Evaluation shared service provides the framework; the domain provides the criteria.
- **Domain-specific runtime assurance signals.** Drift detection, mission coherence monitoring, policy deviation detection, and explainability surfacing are calibrated per domain. The Observability service captures the signals; the domain governance team owns the calibration.

This pattern generalizes. Three initial domains are described below for illustration. Additional domains follow the same pattern.

6.2 Development

Focus: software development lifecycle and engineering augmentation.

Representative use cases:

- Requirements analysis and decomposition
- Architecture decision support
- Code and test generation
- CI/CD pipeline support

- Documentation generation and maintenance

The Development domain consumes the Code Generation Models from the Model Gateway, the Documentation Retrieval sources from the Vector store, and CI/CD integrations from the Tool and API Integration service. Domain-specific tools include source control connectors, build system integrations, and static analysis tooling.

6.3 Infrastructure Operations

Focus: operational state, incident response, and service reliability.

Representative use cases:

- Observability correlation across telemetry sources
- Incident triage and root cause analysis
- Capacity forecasting and resource optimization
- Automated remediation for known incident patterns

The Infrastructure Operations domain consumes the Observability service heavily, the Memory Fabric for incident history, and the Tool and API Integration service for infrastructure control plane access. Trust escalation in this domain typically advances faster than in others because the action space is well-bounded and the evidence base accumulates quickly.

6.4 Cyber Security

Focus: adversarial defense, risk analysis, and trust enforcement.

Representative use cases:

- Threat detection from SIEM telemetry
- Vulnerability prioritization based on environmental context
- Attack path analysis
- Identity risk analysis and anomaly detection

The Cyber Security domain has the strongest constraints on means election. The threat surface implications of authorizing a new tool require careful review. Trust escalation in this domain advances more conservatively than in others; the consequences of error are higher.

6.5 Other Capability Domain Orchestrators

The three named domains above (Development, Infrastructure Operations, Cyber Security) constitute the MVP scope for the Domain AI Ecosystems layer. They are illustrative of the pattern and well-chosen as first deployments because they share an IT-operational character: bounded action spaces, observable feedback loops, and rapid audit evidence accumulation.

The pattern generalizes. The reference architecture explicitly accommodates additional Domain AI Ecosystems instantiated by the enterprise as capability needs emerge. Figure 1 represents these as the *Other Capability Domain Orchestrators* box alongside the three named domains. The box is intentionally not enumerated in MVP scope; an enterprise has substantially more capability areas than three, and overspecifying them in the canonical reference architecture would prematurely commit the framework to specific domain shapes.

Candidate additional domains an enterprise may instantiate over time:

- **Business Operations.** Workflow automation, decision support, process orchestration for non-IT business functions.
- **Engineering and Product Development.** Engineering analysis, design support, requirements traceability, technical knowledge management.
- **Customer-Facing Operations.** Customer service augmentation, support workflow assistance, knowledge surfacing for customer-facing staff.
- **Mission and Program Operations.** Program-specific AI capabilities tailored to mission requirements, often with bounded scope and elevated security controls.
- **Enterprise Knowledge Management.** Cross-domain knowledge retrieval, expert finding, knowledge graph construction.

Each candidate follows the common pattern specified in Section 6.1: bounded scope, shared service consumption, domain-specific agents and tools, domain-specific evaluation criteria, domain-specific runtime assurance signals. None is architecturally special; the common pattern is what makes domain ecosystems composable. The architectural commitment is the pattern itself, not the enumeration of specific instances.

---

7. Layer 3 – AI Runtime and Execution

The AI Runtime and Execution Layer is the foundation on which everything else operates. It is the single point of policy enforcement before real-world consequence. Every action with real-world impact passes through this layer.

7.1 Components

The Runtime comprises:

- **Agent runtimes.** Execution environments for agent operations. Provide isolation, resource bounds, lifecycle management, and instrumentation for the agents instantiated by domain ecosystems.

- **Workflow engines.** Execute multi-step workflows defined by the Orchestrator or by domain agents. Provide checkpointing, recovery, and audit-trail emission across workflow steps.
- **Model endpoints.** The inference infrastructure. Foundation models are served through endpoints accessed via the Model Gateway shared service. The Runtime hosts the endpoints; the Gateway abstracts the access.
- **Tool adapters.** The execution machinery for tool invocations. Adapters mediate between the abstract tool contract (registered in the Schema and Contract Registry) and the concrete tool implementation. Adapters enforce pre-invocation policy checks, capture outcome telemetry, and emit audit evidence.
- **Retrieval pipelines.** Execute retrieval-augmented generation flows. Coordinate between the Vector store, the Memory Fabric, and inference endpoints. Provide retrieval-step audit evidence.
- **Sandboxed execution environments.** Isolated environments for actions with significant blast radius (code execution, system commands, third-party API invocations). The sandbox enforces resource limits, network isolation, and rollback capability.
- **API gateways.** The boundary between the architecture and external systems. Inbound requests are authenticated and authorized; outbound requests are logged and rate-limited. The gateway is the network-layer enforcement point for the architecture's perimeter.
- **Event-driven automation.** The substrate for reactive workflows triggered by observability signals, schedule, or external events. Provides the same policy enforcement and audit emission as request-driven workflows.

7.2 The Sandbox Boundary

The Runtime's architectural commitment is stronger than the bullet list implies. Every action with real-world consequence passes through the Runtime, and every such passage is a policy enforcement point.

This means:

- An agent cannot invoke a tool except through the Runtime. An agent that bypasses the Runtime is operating outside the architecture and produces no audit evidence.
- A tool cannot execute except inside a sandboxed environment with declared resource bounds and an explicit policy decision authorizing the invocation.
- A model inference call cannot occur except through the Model Gateway, which is itself a Runtime-mediated path.
- An external system cannot be invoked except through an API gateway that enforces authorization, rate limiting, and audit emission.

The Runtime is not optional infrastructure; it is the architectural enforcement layer. An architecture that treats the Runtime as a convenience layer (one path among several) is not the architecture this document

describes. The single-path commitment is what makes the audit chain queryable, the policy enforcement reliable, and the sandbox boundary meaningful.

---

8. Cross-Cutting – Governance, Risk, and Assurance

Governance is not a layer in the dependency stack. It is a cross-cutting concern that applies to every layer. Every component above inherits governance constraints; every action above is subject to governance evidence requirements.

8.1 Controls

The Governance, Risk, and Assurance layer provides:

- **Data classification.** Sensitivity classification for data flowing through the architecture. Inherited from request context and propagated through every step. Restricts tool authorization, model selection, and retrieval scope.
- **Access control.** Authorization rules for users, agents, and inter-component invocations. Inherited from the Identity and trust shared service; declared as governance policy.
- **Human-in-the-loop gates.** Trust-escalation-level rules that determine when human approval is required before action. Declared in execution-policy; enforced at the Runtime; mediated by the Human Approval Workflow shared service.
- **Trust escalation model.** Four levels (advisory only, recommend and approve, automated with audit, fully autonomous). Each capability's current level is governance-declared and evidenced by runtime assurance signals.
- **Auditability.** Requirements for what audit evidence must be captured, retained, and queryable. Audit retention policy is declared here; emission happens in the Runtime; storage happens in the Observability service.
- **Explainability requirements.** Standards for what reasoning chains must be surfaceable for review. Required for trust escalation advancement. Declared per domain and per trust level.
- **Model risk management.** Policies for model selection, model lifecycle (introduction, validation, retirement), and substrate substitution. Encompasses both technical risk (drift, behavioral non-determinism) and governance risk (acceptable use, classification appropriateness).
- **Compliance mapping.** Translation of external compliance frameworks (FedRAMP, ITAR, CMMC, sector-specific regulations) into architecture-internal controls. Compliance is enforced at the Runtime; mapping is declared here.
- **Security boundaries.** Network segmentation, enclave boundaries, perimeter enforcement requirements. The architecture's security posture is declared here; enforcement is distributed across the Runtime, API gateways, and the enclave deployment pattern.

- **Acceptable-use policy.** What the architecture may and may not be used for. Declared here; enforced through policy gates at the Orchestrator and Runtime.
- **Evaluation standards.** Quality criteria, outcome scoring rules, and acceptance thresholds. Domain-specific criteria implement these standards; the standards themselves are governance-declared.

8.2 The Declare-Enforce-Evidence Doctrine

The relationship between Governance, Runtime, and Observability is foundational and is stated formally in Section 2 (principle 7) as the *Declare-Enforce-Evidence Doctrine*. This section unpacks the doctrine for architects who need to apply it operationally. Each layer has a distinct role:

- **Governance declares.** Policy authority lives here. Rules are stated, thresholds set, requirements defined. Governance does not directly enforce; it declares what shall be enforced.
- **Runtime enforces.** Policy is applied at execution. Pre-action validation, sandbox enforcement, authorization checks, and resource limits all happen here. The Runtime is the layer that says "no" or "yes" to a specific action based on governance-declared rules.
- **Observability evidences.** Enforcement is recorded. Audit evidence is captured, performance metrics tracked, runtime assurance signals emitted. Observability does not declare or enforce; it evidences that declaration and enforcement happened.

The doctrine's three-way split is essential. An architecture that lets governance be its own enforcement layer is structurally unable to scale; governance becomes a bottleneck. An architecture that lets runtime declare its own policy is structurally unable to be audited; policy becomes implicit in code. An architecture without an observability layer that captures enforcement evidence is structurally unable to demonstrate compliance; enforcement is unprovable.

The Declare-Enforce-Evidence Doctrine is invoked throughout this document. When the Orchestrator validates a request (Section 4), it does so against governance-declared rules. When the Runtime authorizes a tool invocation (Section 7), it enforces those declarations. When the Observability service captures audit evidence (Section 9.4), it evidences both. Each layer's distinct role is what makes the audit chain queryable end-to-end and the governance posture reviewable.

---

9. Interaction Patterns

Components describe what exists. Patterns describe how the components interact for specific scenarios. The patterns below cover the most common interaction shapes; an architect designing for a specific use case extends these patterns rather than inventing from scratch.

9.1 Request Lifecycle

The canonical pattern. A user submits a request; the request flows through the architecture; an outcome returns; audit evidence accumulates at every step.

Figure 2 illustrates the lifecycle.

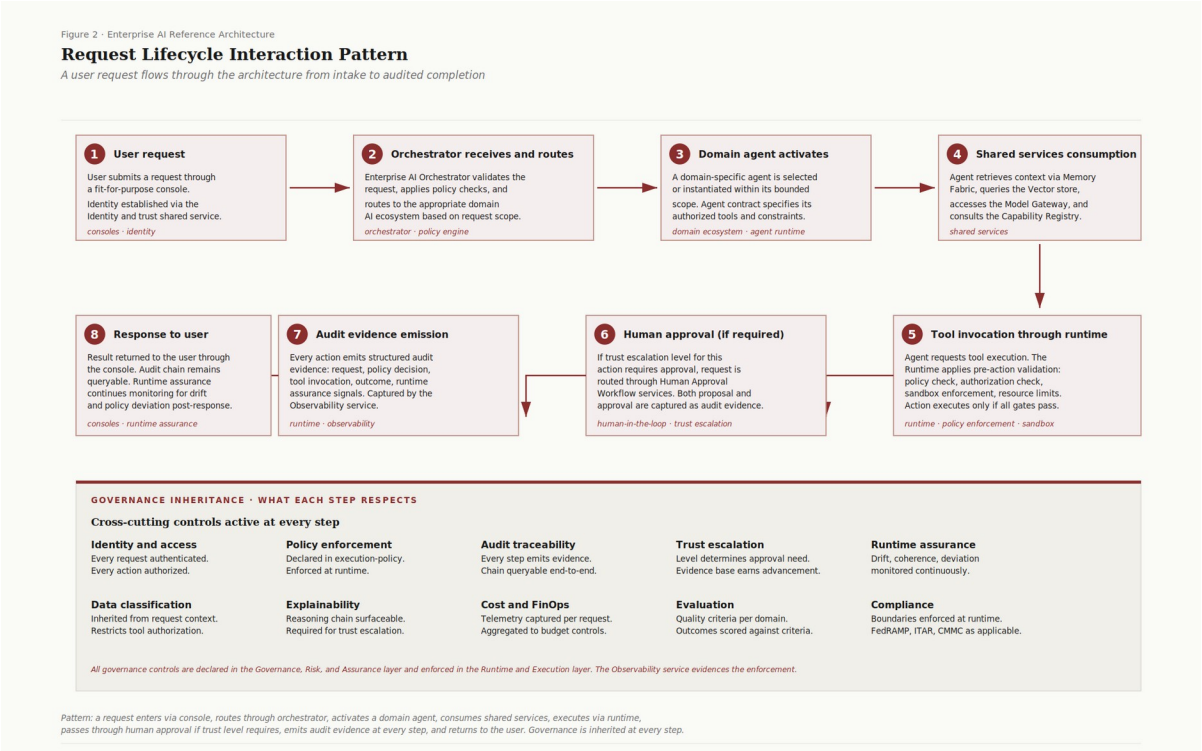

Figure 2. Request Lifecycle Interaction Pattern. A user request flows through the architecture from intake to audited completion.

The sequence:

1. **Request enters.** User submits through a fit-for-purpose console. Identity established via the Identity and Trust shared service.

2. **Orchestrator routes.** Request validated, policy-checked, routed to the appropriate domain ecosystem.

3. **Domain agent activates.** Within the domain, a specialist agent is selected or instantiated. Agent contract specifies authorized tools and constraints.

4. **Shared services consulted.** Agent retrieves context, accesses the Model Gateway, consults the Capability Registry.

5. **Tool invocation through Runtime.** Agent requests tool execution. Runtime applies pre-action validation: policy check, authorization check, sandbox enforcement, resource limits. Action executes only if all gates pass.

6. **Human approval if required.** If the trust escalation level for this action requires approval, the request routes through the Human Approval Workflow service. Both proposal and approval captured as audit evidence.

7. **Audit evidence emission.** Every step emits structured audit evidence. Captured by the Observability service.

8. **Response to user.** Result returned through the console. Audit chain remains queryable. Runtime assurance continues monitoring post-response.

Governance controls (identity, policy, audit, trust escalation, runtime assurance, classification, explainability, cost, evaluation, compliance) apply at every step. The cross-cutting Governance layer declares them; the Runtime enforces them; the Observability service evidences them.

9.2 Cross-Domain Coordination

When a request requires action across multiple domains, the Orchestrator coordinates rather than letting domains invoke each other directly.

Pattern: a request entering one domain that needs information or action from another domain returns to the Orchestrator, which routes the secondary request to the target domain. The Orchestrator maintains the workflow context, the audit trail, and the policy state across domain boundaries.

Domains do not call each other. Cross-domain invocations always pass through the Orchestrator. This preserves the architectural property that the Orchestrator is the single authority for cross-domain coordination and the single point where cross-domain policy is enforced.

9.3 Human Approval Cycle

Trust escalation levels 1 (advisory only) and 2 (recommend and approve) require human authority before action. Level 3 (automated with audit) and 4 (fully autonomous) do not, but level 3 retains audit emission and level 4 retains runtime assurance monitoring.

Pattern for levels 1 and 2:

- Agent produces a recommendation or proposed action.
- Recommendation is routed to the Human Approval Workflow service.
- Designated human authority (per the governance-declared approval matrix) reviews.
- Approval, rejection, or modification is captured.
- If approved, the action proceeds through the Runtime with the standard policy enforcement.
- Both the original recommendation and the approval (or rejection) are captured as audit evidence.

The Human Approval Workflow service is a shared service; domains do not build their own. The approval matrix is governance-declared; domains do not set their own thresholds.

9.4 Audit Evidence Aggregation

Audit evidence emitted by the Runtime and the Orchestrator is captured by the Observability service. Audit aggregation across the architecture follows a layered pattern:

- **Per-action audit records.** Captured at the moment of action. Include the request, the policy decision, the agent context, the tool invocation, the outcome, and the runtime assurance signals.
- **Per-workflow audit chains.** Multi-step workflows produce audit chains that link individual action records into a queryable sequence. The Orchestrator emits workflow-level audit records that reference the per-action records.
- **Per-domain audit aggregations.** Domain ecosystems produce domain-level audit summaries that aggregate per-action and per-workflow evidence. Used for domain governance reviews.
- **Cross-domain audit aggregations.** The Observability service produces cross-domain summaries for enterprise governance review. Used for trust escalation reviews and compliance reporting.

The audit chain is queryable end-to-end: an enterprise governance reviewer can trace a high-level compliance question down to specific action evidence without losing accountability at any layer.

9.5 Means Election

A new tool is added to the Capability Registry as a mechanism. To become a means, it requires election. The pattern:

- The tool's introducing party (a domain owner, a capability owner, a procurement decision) proposes election.
- The proposal identifies the scope of intended use: which domain, which agents, which trust levels, which classifications.
- The means election review (a governance forum, declared in Modus Primus Section 9.6) evaluates the proposal against three axes:

- **Mission advancement.** Does the tool advance the mission of the proposing domain?

- **Morals consistency.** Is the tool's election consistent with security, safety, and acceptable-use constraints?

- **Inventory justification.** Does the existing means inventory justify the addition? Is anything being retired in exchange?

- The review approves, modifies, or rejects the proposal. The decision is recorded in the Capability Registry.

The pattern is what prevents silent capability creep. Every newly procured tool may be operationally available; no tool becomes part of the system's disposition without explicit governance election. The friction is the feature.

---

10. Federation Across Enclaves

Single-enclave deployments are described above. Real enterprises operating in regulated industries or defense IT do not deploy in a single enclave. The federation pattern is foundational, not optional.

Figure 3 shows the federation deployment view.

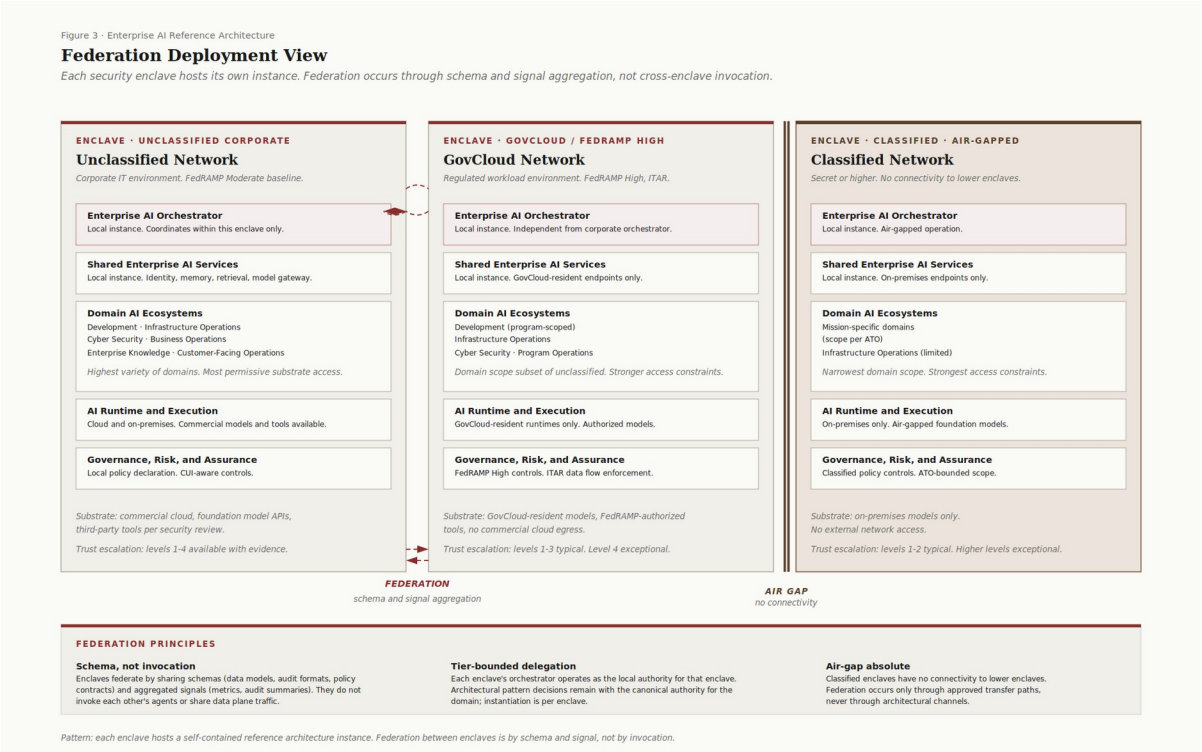

Figure 3. Federation Deployment View. Each security enclave hosts its own instance. Federation occurs through schema and signal aggregation, not cross-enclave invocation.

10.1 The Enclave Constraint

Security enclaves are bounded environments with their own authorization scope, their own compliance posture, and their own data flow restrictions. Common enclaves in an aerospace-defense enterprise:

- **Unclassified Corporate.** The FedRAMP Moderate baseline corporate environment. Hosts the broadest variety of domains and the most permissive substrate access.
- **GovCloud / FedRAMP High.** Regulated workload environments. ITAR data flow controls, restricted model selection, GovCloud-resident services.

- **Classified.** Air-gapped or strictly bounded networks. On-premises substrate only, narrowest domain scope, strongest access controls.
- **Program-specific enclaves.** Mission-bounded environments scoped to specific programs. ATO-defined scope, program-specific policy controls.

Each enclave hosts its own complete instance of the reference architecture. The Orchestrator, Shared Services, Domain Ecosystems, Runtime, and Governance controls are all instantiated per enclave. There is no shared substrate that crosses enclave boundaries.

10.2 Federation Mechanics

Federation between enclaves occurs through two channels:

- **Schema federation.** Enclaves share schemas: data models, audit formats, policy contracts, agent contract specifications, federation event schemas. Schema versioning is centrally coordinated; instantiations are per-enclave.
- **Signal aggregation.** Enclaves share aggregated signals: audit summaries, metrics, telemetry rollups, compliance evidence. Raw operational data is not shared across enclaves; aggregations are.

Federation does **not** occur through:

- Cross-enclave invocation. Agents in one enclave do not invoke agents in another enclave.
- Data plane traffic. Operational data does not flow across enclave boundaries.
- Shared runtime. Each enclave has its own Runtime, with its own policy enforcement, its own sandbox boundaries, and its own audit emission.

The constraint preserves enclave sovereignty. Each enclave's compliance posture is self-contained; cross-enclave federation does not compromise any enclave's controls.

10.3 Air-Gap Boundaries

Classified enclaves have absolute connectivity boundaries with lower enclaves. The reference architecture does not provide an architectural channel across the boundary. Federation between classified enclaves and lower enclaves occurs through approved transfer paths (cross-domain solutions, manual data transfer with review, controlled releases), never through architectural channels.

The architectural pattern for classified enclaves is intentionally narrower than for unclassified or GovCloud enclaves:

- Substrate is on-premises only. No external network access. No commercial cloud egress.
- Domain ecosystems are mission-specific and bounded by ATO scope.
- Trust escalation typically operates at levels 1 to 2. Higher levels require specific governance authorization.

- Runtime assurance is calibrated tighter; thresholds for human intervention are lower.

The architecture is consistent across enclaves; the scope and constraints differ.

---

11. Adoption Sequencing

A reference architecture without adoption guidance is paperware. The sequencing below identifies which components an enterprise builds first, what advances to subsequent phases, and what triggers progression.

11.1 Phase 1 – Foundation

Build the substrate that everything else depends on. Defer domain-specific capability until the substrate is operational.

Components built first:

- The AI Runtime and Execution Layer at minimum viable scope. Agent runtimes, tool adapters, sandboxed execution, API gateways. Without the Runtime, no other component can operate.
- The Identity and Trust shared service. Integrated with the enterprise IAM. Required for every other component's authorization model.
- The Observability and Telemetry service. Required to capture audit evidence from the Runtime and demonstrate compliance.
- The Governance, Risk, and Assurance layer in foundational form. Initial policy declarations, audit retention rules, trust escalation level definitions.
- The Capability Registry. Even at small scale, the registry establishes the means-versus-mechanism discipline early.

Phase 1 advances when:

- The Runtime executes simple actions with policy enforcement and audit emission. Demonstrated end-to-end.
- Identity flows through the architecture correctly.
- Observability captures and surfaces audit evidence. Demonstrated queryable.
- Initial governance policies are declared and enforced.

MVP-Conformant Deployment

A deployment is MVP-conformant when all Phase 1 components are operational at minimum viable capability and the Phase 1 advancement criteria above are met. Specifically, an MVP-conformant deployment includes the AI Runtime and Execution Layer, the Identity and Trust shared service, the

Observability and Telemetry service, the Governance Risk and Assurance layer in foundational form, and the Capability Registry. These five components are non-negotiable; an architecture missing any of them is not MVP-conformant regardless of what other components it contains.

Deferrals are explicit. MVP scope excludes additional Domain AI Ecosystems beyond the first, federation infrastructure across enclaves, advanced trust escalation levels (3 and 4), the full set of Shared Enterprise AI Services beyond the foundational identity and observability subset, and cross-domain workflow capabilities. These are not eliminated; they are sequenced into Phase 2 and Phase 3. An enterprise that attempts to deploy them in Phase 1 is overcommitting; an enterprise that defers them past Phase 3 is undercommitting.

The boundary between MVP scope and deferred scope is the architectural definition of phase completion. An architect or governance reviewer can determine MVP-conformance by inspection of the five non-negotiable components and verification against the four advancement criteria. The table below specifies each component, its typical organizational owner, its minimum capability for MVP-conformance, and its maturity expansion path through Phase 2 and Phase 3.

Component	Owner role	Minimum capability (MVP)	Maturity expansion
AI Runtime and Execution	Platform Engineering	Agent runtimes, tool adapters, sandboxed execution, API gateways for a small set of authorized tools	Add workflow engines, retrieval pipelines, event-driven automation, and broader tool inventory in Phase 2
Identity and Trust	Cybersecurity / Enterprise IAM	Integration with enterprise IAM, identity context flowing through every audit record	Expand to fine-grained agent identity, capability-bound authorization, and federated identity across enclaves
Observability and Telemetry	IT Operations	Audit evidence capture, basic operational metrics, queryable audit chain end-to-end	Add runtime assurance signals (drift, mission coherence, policy deviation), cost telemetry aggregation, evaluation reporting
Governance, Risk, and Assurance	Enterprise Architecture / Data and AI Governance	Initial policy declarations, audit retention rules, trust	Add compliance mapping for additional frameworks, advanced

		escalation level definitions for levels 1 and 2	trust escalation criteria for levels 3 and 4, cross-domain governance reviews
Capability Registry	Enterprise Architecture	Catalog of authorized tools, models, and retrieval sources; distinction between mechanism and means	Add means election workflow, schema versioning, federation across enclaves
Enterprise AI Orchestrator (Phase 2 entry)	Platform Engineering	Routes requests to the single initial Domain Ecosystem; basic policy enforcement at coordination layer	Add cross-domain orchestration, workflow coordination, cost and performance governance
First Domain AI Ecosystem (Phase 2 entry, recommended: Infrastructure Operations)	IT Operations	Domain-specific agents in advisory mode (trust level 1), domain-specific evaluation criteria, domain-specific runtime assurance signals	Advance to trust level 2 and 3 with evidence; add additional domains in Phase 3
Remaining Shared Services (Phase 2)	Platform Engineering / Data and AI Governance	Memory Fabric, Vector and Knowledge Retrieval, Model Gateway, Tool/API Integration, Schema Registry, Evaluation, Human Approval Workflow, Cost telemetry	Federation-aware service instances per enclave in Phase 3

The table is illustrative of typical organizational ownership; specific role assignments are enterprise-specific and should be confirmed during Phase 0 chartering. The capability descriptions are minimum viable; an enterprise may exceed them but should not deploy below them.

11.2 Phase 2 – First Domain

Build the first domain ecosystem against the established substrate. The choice of first domain matters; the recommendation is Infrastructure Operations.

Why Infrastructure Operations as first domain:

- The action space is well-bounded. Operational actions have clear definitions, clear acceptance criteria, and known failure modes.
- The audience (IT operations staff) is technically sophisticated and operationally pragmatic. Less philosophical resistance, more practical engagement.
- Trust escalation advances faster in this domain than in others because evidence accumulates quickly and the consequences of error are recoverable.
- The Infrastructure Operations domain produces immediate value (incident triage, observability correlation, capacity forecasting) that justifies further investment.

Components built in Phase 2:

- The Enterprise AI Orchestrator at minimum viable scope. Routes requests to the single initial domain. Will expand as additional domains come online.
- The remaining Shared Enterprise AI Services: Memory Fabric, Vector and Knowledge Retrieval, Model Gateway, Tool/API Integration, Schema Registry, Evaluation, Human Approval Workflow, Cost telemetry.
- The Infrastructure Operations domain ecosystem. Domain agents, domain-specific tools, domain-specific evaluation criteria, domain-specific runtime assurance signals.

Phase 2 advances when:

- The first domain operates at level 1 (advisory only) with positive evaluation feedback over a defined period.
- The Orchestrator handles cross-domain coordination correctly (even if there is only one domain initially).
- Means election has been exercised at least once for a new tool added to the domain.
- The audit chain is queryable end-to-end for a representative incident scenario.

11.3 Phase 3 – Federation and Scale

Add additional domains. Address the enclave deployment pattern. Mature trust escalation.

Components built in Phase 3:

- Additional Domain AI Ecosystems. Cyber Security is a frequent second domain; Development is a frequent third. Each follows the common pattern.
- Federation infrastructure if multi-enclave operation is required. The schema federation, signal aggregation, and per-enclave instance pattern.
- Cross-domain workflow capabilities. The Orchestrator's domain orchestration responsibilities exercise here.
- Trust escalation advancement for proven Phase 2 capabilities. Level 1 → level 2 → level 3 progression as evidence accumulates.

Phase 3 advances when:

- Multiple domains operate at appropriate trust levels with documented evidence.
- Cross-domain coordination is demonstrated for representative scenarios.
- Federation across at least one enclave boundary is operational (if applicable to the enterprise).
- Trust escalation reviews are a regular governance forum, not an exceptional event.

---

12. Standards Alignment

The reference architecture is consistent with established systems engineering and architecture-description conventions. The mapping below identifies the principal correspondences; the architecture does not claim to be a strict implementation of any single standard.

Architecture concept	External standard analogue	Notes
Layered decomposition	ISO/IEC/IEEE 42010 architecture description	Coordination, Capability, Foundation, Cross-cutting Governance
Component view	TOGAF capability viewpoint	Components, responsibilities, boundaries
Interaction patterns	UAF / DoDAF operational viewpoints	Request lifecycle, cross-domain coordination, audit aggregation
Federation deployment view	TOGAF federation; ISO/IEC/IEEE 42010 architecture frameworks	Per-enclave instances, schema and signal federation
Foundational principles	INCOSE architectural principles practice	Decision filters for design choices
Adoption sequencing	INCOSE acquisition lifecycle; SAFe enterprise adoption	Phased rollout with explicit advancement criteria

Where this architecture's terminology differs from a standard's, this architecture's terminology is authoritative within its own scope. Implementation against specific standards (e.g., a TOGAF-compliant architecture description) is enterprise-specific and falls outside the canonical reference architecture.

References

References use Harvard-style citations with hanging indents.

INCOSE (2023). *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, 5th edition. International Council on Systems Engineering. Hoboken, NJ: Wiley.

ISO/IEC/IEEE (2022). *ISO/IEC/IEEE 42010:2022, Software, systems and enterprise, Architecture description*. International Organization for Standardization, Geneva.

Longmire, J. (2026). *Portable Agent Harness Architecture: A Capability-Centric Framework for Governed AI Ecosystems in Sovereignty-Bounded Enterprises*, Revision 2.2. Zenodo. https://doi.org/10.5281/zenodo.20112631

Longmire, J. (2026). *Modus Primus: Engineering Specification for AI Architecture (PAHA Companion)*, Version 1.1. Zenodo. https://doi.org/10.5281/zenodo.20113785

The Open Group (2022). *TOGAF Standard, 10th Edition*. The Open Group, San Francisco.

Appendix A – Glossary

Agent. A bounded execution unit within a domain ecosystem. Agent contracts (per Modus Primus Appendix E) specify role bindings, capability authorizations, and audit obligations.

Capability Registry. The shared service that catalogs all available tools, models, retrieval sources, and agents in the architecture. Distinguishes between operationally available mechanisms and purposively elected means.

Domain AI Ecosystem. A bounded scope within Layer 2B where domain-specific agents, tools, and workflows operate. Each domain has its own scope, evaluation criteria, and operational concerns.

Enclave. A security-bounded environment with its own authorization scope, compliance posture, and data flow restrictions. Each enclave hosts its own instance of the reference architecture.

Enterprise AI Orchestrator. The Layer 1 coordination plane. Routes requests, coordinates workflows, enforces cross-domain policy, and manages agent lifecycles. The Orchestrator coordinates; it does not execute.

Federation. The pattern by which separate enclaves share schemas and aggregated signals without sharing data plane traffic or invoking each other's components.

Means. A mechanism that has been purposively elected by the Governance layer as authorized for use within specific scopes. Means are part of the system's disposition.

Mechanism. Operationally available capacity (a tool, a model, an integration) that is listed in the Capability Registry but not yet authorized for use. Mechanisms become means only through election.

Runtime Assurance. Continuous monitoring of drift, mission coherence, policy deviation, and explainability as architectural commitments. Produces validation evidence between formal review milestones.

Shared Enterprise AI Services. The Layer 2A common infrastructure consumed by every domain ecosystem. Identity, memory, retrieval, model gateway, evaluation, observability, capability registry, schema registry, human approval, cost telemetry.

Trust Escalation. The four-level model (advisory only, recommend and approve, automated with audit, fully autonomous) for the autonomy granted to an AI capability. Each advancement requires governance approval grounded in evidence.

Appendix B – Mapping to PAHA and Modus Primus

The reference architecture decomposition maps onto PAHA and Modus Primus as follows.

Reference architecture element	PAHA element	Modus Primus element
Enterprise AI Orchestrator (Layer 1)	Meta-harness coordination services	Orchestration Layer (Section 3.4, Appendix B.5)
Identity and Trust (Shared Service)	Trust escalation primitive	Cross-cutting governance (Section 8)
Memory Fabric (Shared Service)	Meta-harness memory service	memory.md (Section 2.3, B.3.4) and Observability infrastructure (B.10)
Model Gateway (Shared Service)	Substrate adapter abstraction	Cognitive Engine Layer (Section 3.3, Appendix B.4)
Tool/API Integration (Shared Service)	Capability registry	Mechanism Layer (Section 3.6, Appendix B.7)
Capability Registry (Shared Service)	Capability registry with purposive election	Means and mechanisms selection (Section 2, Appendix C)
Schema and Contract Registry (Shared Service)	Federation schema services	Federation schema (Section 4)

Observability and Telemetry (Shared Service)	Audit infrastructure	Observability infrastructure (Section 3.9, Appendix B.10)
Evaluation and Testing (Shared Service)	V&V infrastructure	V&V Specialization (Section 7)
Human Approval Workflow (Shared Service)	Trust escalation enforcement	Trust escalation model (Sections 2.5, 3.7.1)
Domain AI Ecosystems (Layer 2B)	Composable agents and fit-for-purpose consoles	Agent Layer (Section 3.5, Appendix B.6)
AI Runtime and Execution (Layer 3)	Bounded execution primitive	Execution Runtime (Section 3.7.2, Appendix B.8.2)
Governance, Risk, and Assurance (Cross-cutting)	Centralized governance primitive	Execution Governance (Section 3.7, Appendix B.8)

This mapping is the authoritative cross-reference. The reference architecture's component decomposition is a faithful realization of PAHA's architectural primitives in the form that an enterprise architect can directly use, and a faithful realization of Modus Primus's engineering specifications in the form that an implementation team can directly build.

This is version 1.1 of the Enterprise AI Reference Architecture, incorporating review refinements over the v1.0 initial publication. Refinements in v1.1 include naming the Declare-Enforce-Evidence Doctrine (Section 2 principle 7 and Section 8.2), tightening the Orchestrator's coordination claim to workflow-level mediation (Section 4), introducing MVP-conformant deployment criteria and a component-to-owner-to-capability adoption table (Section 11.1), and aligning Section 6.5 with the Figure 1 Other Capability Domain Orchestrators framing. The architecture is intended to support adoption across enterprise contexts with varying levels of existing engineering practice and varying enclave configurations. The reference architecture commitments stated here apply to AI architecture specifically; enterprise practice integration occurs through enterprise-specific addenda.